List Comprehensions

[expression for item in iterable if condition] – the idiomatic way to build a new list.

Pattern table

Goal	Pattern
map	`[x * 2 for x in xs]`
filter	`[x for x in xs if x > 0]`
map + filter	`[x ** 2 for x in xs if x % 2]`
if / else	`[x if x > 0 else 0 for x in xs]`
nested loops	`[(a, b) for a in A for b in B]`
flatten	`[x for row in m for x in row]`
list of lists	`[[0] * 3 for _ in range(2)]`
call a function	`[len(w) for w in words]`

Examples

```
nums = [-2, -1, 0, 1, 2]
print([n * 2 for n in nums])
print([n for n in nums if n > 0])
print([n if n > 0 else 0 for n in nums])
words = ["apple", "fig", "kiwi"]
print([w.upper() for w in words if len(w) > 3])
```

OUTPUT

```
[-4, -2, 0, 2, 4]
[1, 2]
[0, 0, 0, 1, 2]
['APPLE', 'KIWI']
```

Nested loops read left → right

```
pairs = [(a, b) for a in "xy" for b in (1, 2)]
print(pairs)

matrix = [[1, 2, 3], [4, 5, 6]]
flat = [x for row in matrix for x in row]
print(flat)
cols = [[r[i] for r in matrix] for i in range(3)]
print(cols)                              # transpose
```

OUTPUT

```
[('x', 1), ('x', 2), ('y', 1), ('y', 2)]
[1, 2, 3, 4, 5, 6]
[[1, 4], [2, 5], [3, 6]]
```

enumerate, zip and the walrus

```
names, ages = ["Ann", "Bob"], [31, 25]
print([f"{i}:{n}" for i, n in enumerate(names, 1)])
print([f"{n}={a}" for n, a in zip(names, ages)])
print([y for x in [1, 2, 3] if (y := x * x) > 1])
```

OUTPUT

```
['1:Ann', '2:Bob']
['Ann=31', 'Bob=25']
[4, 9]
```

Recipes on strings

```
text = "The quick brown fox"
print([w[0] for w in text.split()])
print([ch for ch in text if ch in "aeiou"])
print([int(d) for d in "2026"])
```

OUTPUT

```
['T', 'q', 'b', 'f']
['e', 'u', 'i', 'o', 'o']
[2, 0, 2, 6]
```

Equivalent loop

```
[E for x in X if C]

result = []
for x in X:
    if C:
        result.append(E)
```

Pitfall: shared rows

```
bad = [[0] * 2] * 2          # same inner list twice!
bad[0][0] = 1
print(bad)
good = [[0] * 2 for _ in range(2)]
good[0][0] = 1
print(good)
```

OUTPUT

```
[[1, 0], [1, 0]]
[[1, 0], [0, 0]]
```

- Don't use one for side effects: `[print(x) for x in xs]` builds a useless list.
- More than two `for`s or a long condition? Use a loop or a helper function.
- It builds the **whole** list in memory – iterating once over big data? Use `(...)`.

TIP `[f(x) for x in xs]` is usually faster than the same `for` + `append` loop, and easier to scan.